

# ĐỀ CƯƠNG ÔN FINAL

CSC14003 - Nhập môn Trí tuệ nhân tạo

Bám slide giảng viên + đối chiếu 3 bộ đề cuối kỳ 2021-2024

| Ưu tiên | Chủ đề                               | Tần suất trong 3 bộ đề | Cách học chắc điểm                     |
|---------|--------------------------------------|------------------------|----------------------------------------|
| 1       | ID3 / Decision Tree                  | 3/3                    | Thuộc công thức; tự tính trọn một cây  |
| 1       | PL/FOL: CNF, unification, resolution | 3/3                    | Luyện biến đổi và ghi từng clause      |
| 2       | Minimax + Alpha-Beta                 | Có ở các đề gần đây    | Duyệt đúng thứ tự, ghi $\alpha/\beta$  |
| 2       | Naive Bayes                          | Có ở các đề gần đây    | Prior, likelihood, Laplace             |
| 3       | CSP + AC-3                           | 1/3                    | MRV/LCV, domain sau mỗi bước           |
| 4       | Perceptron / MLP                     | Chưa thấy trong 3 đề   | Nắm forward, delta, update             |
| 5       | Search + PEAS + local search         | Nền tảng môn           | So sánh thuật toán và điều kiện tối ưu |

## Cách dùng tài liệu

- Học theo thứ tự ưu tiên ở bảng trên. Mỗi chủ đề: đọc slide chỉ định, làm lại ví dụ, rồi giải đề cũ không nhìn lời giải.
- Đề cương này để hiểu quy trình. File cheatsheet đi kèm để ôn công thức và bấm ngay trước giờ thi.

Quy ước: phần 'Bấy' là lỗi dễ mất điểm; phần 'Khi làm bài' là thứ tự trình bày nên ghi ra giấy.

## Nguồn nội bộ

Lecture 1-11 trong thư mục slide; syllabus CSC14003; đề cuối kỳ 2021-2022, 2022-2023, 2023-2024 trong thư mục finalexam.

# 1. ID3 / Decision Tree | ƯU TIÊN CAO NHẤT - xuất hiện 3/3 đề

Đọc ở slide: Lecture 10 - Basic Machine Learning, trang 33-57; entropy/Information Gain trang 40-54; quiz trang 57.

## Công thức lỗi

$$\begin{aligned} \text{Entropy}(S) &= -\sum_c p(c) \log_2 p(c) \\ \text{AverageEntropy}(S,A) &= \sum_v |S_v|/|S| \cdot \text{Entropy}(S_v) \\ \text{IG}(S,A) &= \text{Entropy}(S) - \text{AverageEntropy}(S,A) \end{aligned}$$

## Gain Ratio (từng xuất hiện trong đề 2021-2022)

$$\begin{aligned} \text{SplitInfo}(S,A) &= -\sum_v |S_v|/|S| \cdot \log_2(|S_v|/|S|) \\ \text{GainRatio}(S,A) &= \text{IG}(S,A) / \text{SplitInfo}(S,A) \end{aligned}$$

## Quy trình dựng cây

- 1) Tính entropy của tập hiện tại.
- 2) Với từng thuộc tính, chia tập theo từng giá trị; tính entropy của từng nhánh.
- 3) Lấy trung bình có trọng số để ra AverageEntropy; suy ra IG hoặc Gain Ratio.
- 4) Chọn thuộc tính có chỉ số lớn nhất làm node; lặp trên từng nhánh.
- 5) Mỗi đường root -> leaf cho một luật IF-THEN.

## Điều kiện dừng

- Tất cả mẫu cùng nhãn: trả leaf có nhãn đó.
- Hết thuộc tính: trả nhãn plurality/majority của tập hiện tại.
- Subset rỗng: dùng plurality/majority của tập cha.

## Bẫy

- Trọng số nhánh là  $|S_v|/|S|$ , không phải chia đều.
- $0 \log(0)$  được quy ước bằng 0.
- IG thiên vị thuộc tính có nhiều giá trị; Gain Ratio hiệu chỉnh thiên vị này.

## Khung trình bày ăn điểm

| Bước           | Phải ghi trên bài thi                                                |
|----------------|----------------------------------------------------------------------|
| Entropy gốc    | Đếm số mẫu từng class -> $p(c)$ -> thay vào công thức                |
| Mỗi thuộc tính | Liệt kê các subset $S_v$ , entropy từng subset, trọng số $ S_v / S $ |
| So sánh        | Lập bảng IG hoặc Gain Ratio của tất cả thuộc tính còn lại            |
| Cây và luật    | Vẽ node/edge rõ giá trị; mỗi root-to-leaf là một rule                |
| Mẫu mới        | Đi từ root theo giá trị thuộc tính; tới leaf để lấy class            |

Tự kiểm: nếu node con thuần nhất thì entropy = 0. Nếu một thuộc tính tách hoàn hảo toàn bộ class thì AverageEntropy = 0 và IG = Entropy(S).

# 2. Naive Bayes | ƯU TIÊN CAO

Đọc ở slide: Lecture 10, trang 62-75; Laplace trang 69-72; quiz trang 75.

Bayes:  $P(C|X) = P(X|C)P(C)/P(X)$ . Giả định naive: các đặc trưng độc lập có điều kiện khi biết class.

$$\text{score}(C) = P(C) \cdot \prod_i P(x_i|C)$$

Chọn class có score lớn nhất. Khi chỉ so sánh class, không cần tính mẫu số  $P(X)$ .

Laplace correction cho đặc trưng rời rạc:

$$P(x_i=v|C) = (\text{count}(v,C)+1) / (\text{count}(C)+k), \text{ với } k \text{ là số giá trị có thể của thuộc tính.}$$

## Quy trình

- 1) Lập bảng đếm class và từng giá trị thuộc tính theo class.
- 2) Prior:  $P(C) = \text{count}(C)/N$ .
- 3) Likelihood: tính  $P(x_i|C)$  cho đúng class; dùng Laplace nếu để yêu cầu hoặc có xác suất 0.
- 4) Nhân prior với toàn bộ likelihood; so sánh score.
- 5) Có thể dùng log-score để biến tích thành tổng khi số đặc trưng lớn.

Mẫu trình bày:  $\text{score}(C1) = P(C1) \times P(x1|C1) \times \dots$ ;  $\text{score}(C2) = \dots$ ; vì  $\text{score}(C1) > \text{score}(C2)$  nên dự đoán C1. Ghi đủ phân số trước khi làm tròn.

## Bẫy

- Không đảo  $P(x|C)$  thành  $P(C|x)$ .
- Một conditional bằng 0 làm cả tích bằng 0; Laplace xử lý việc này.
- Mẫu số Laplace dùng  $\text{count}(C)+k$ , không dùng  $N+k$ .
- Không bỏ prior trừ khi để nói prior các class bằng nhau.
- Nếu để cho số liệu đã làm tròn, không cộng 1 lần nữa.

## ID3 hay Naive Bayes?

| Tiêu chí      | ID3                       | Naive Bayes                   |
|---------------|---------------------------|-------------------------------|
| Sân phẩm      | Cây và luật để giải thích | Xác suất/score theo class     |
| Tính toán lỗi | Entropy, IG/Gain Ratio    | Prior, likelihood, Laplace    |
| Lỗi hay gặp   | Sai trọng số subset       | Sai chiều điều kiện, bỏ prior |

## 3. Propositional Logic: CNF, Resolution, Chaining | ƯU TIÊN CAO NHẤT

Đọc ở slide: Lecture 7 - Logical Agents. Entailment trang 19-22 và 27-32; CNF/resolution trang 44-54; FC/BC quiz trang 83; DPLL trang 84.

## Khái niệm

$KB \models \alpha$  nghĩa là trong mọi model mà KB đúng thì  $\alpha$  cũng đúng. Resolution thường chứng minh bằng phản chứng: kiểm tra  $KB \wedge \neg\alpha$  có dẫn đến mâu thuẫn hay không.

## Các tương đương phải thuộc

$P \rightarrow Q \equiv \neg P \vee Q$   
 $P \leftrightarrow Q \equiv (\neg P \vee Q) \wedge (\neg Q \vee P)$   
 $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ ;  $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$   
 $P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$

## Refutation resolution

- 1) Lập  $KB \wedge \neg\text{query}$ .
- 2) Khử  $\leftrightarrow$  và  $\rightarrow$ .
- 3) Đẩy  $\neg$  vào trong; khử phủ định kép.
- 4) Phân phối  $\vee$  qua  $\wedge$  để ra CNF.
- 5) Tách thành các clause.
- 6) Resolve hai clause có literal đối ngẫu.
- 7) Ra clause rỗng  $\square \rightarrow KB \models \text{query}$ .

## Resolution rule

$(A \vee P), (B \vee \neg P) \Rightarrow (A \vee B)$

## Bẫy

- Phải thêm  $\neg\text{query}$ , không thêm query.
- Clause rỗng mới là chứng cứ mâu thuẫn.
- Đánh số clause; ghi hai clause cha mỗi bước.
- Không resolve hai literal cùng dấu.
- CNF là AND của các clause; mỗi clause là OR của literal.

## Forward chaining và backward chaining

| Phương pháp       | Bắt đầu        | Dừng                             | Ghi nhớ                         |
|-------------------|----------------|----------------------------------|---------------------------------|
| Forward chaining  | Facts trong KB | Query xuất hiện hoặc fixed point | Data-driven; rule Horn          |
| Backward chaining | Query          | Về facts hoặc thất bại           | Goal-driven; tránh lặp mục tiêu |

Nếu forward chaining dừng ở fixed point mà chưa có query, chỉ kết luận 'không suy ra được bằng thủ tục này'; đừng tự ý kết luận query sai nếu ngữ nghĩa bài không cho phép.

## 4. First-Order Logic | CNF + UNIFICATION + RESOLUTION

Đọc ở slide: Lecture 8. Dịch FOL trang 40; UI/EI trang 42-45; GMP/unification trang 48-57; FC trang 69; BC trang 80-81; CNF/Skolem trang 83-86; resolution trang 87-92.

### Dịch ngôn ngữ

| Cụm từ                | Ký hiệu / mẫu                                                                     |
|-----------------------|-----------------------------------------------------------------------------------|
| mọi A đều là B        | $\forall x (A(x) \rightarrow B(x))$                                               |
| có ít nhất một A là B | $\exists x (A(x) \wedge B(x))$                                                    |
| không có A nào là B   | $\forall x (A(x) \rightarrow \neg B(x))$ hoặc $\neg \exists x (A(x) \wedge B(x))$ |
| chỉ A mới là B        | $\forall x (B(x) \rightarrow A(x))$ ; chú ý chiều kéo theo                        |

### Quy trình FOL $\rightarrow$ CNF

- 1) Khử  $\leftrightarrow$ , rồi khử  $\rightarrow$ .
- 2) Đẩy  $\neg$  vào trong qua lượng từ:  $\neg \forall x P \equiv \exists x \neg P$ ;  $\neg \exists x P \equiv \forall x \neg P$ .
- 3) Standardize apart: đổi tên biến ở các mệnh đề khác nhau.
- 4) Skolemize  $\exists$ .
- 5) Bỏ  $\forall$  ngầm định.
- 6) Phân phối  $\vee$  qua  $\wedge$ ; tách clause.

### Skolem

- $\exists x P(x) \rightarrow P(c)$ : dùng Skolem constant mới.
- $\forall y \exists x P(x, y) \rightarrow P(f(y), y)$ : Skolem function nhận các biến  $\forall$  đang bao ngoài.
- Hai lượng  $\exists$  độc lập không được dùng chung một Skolem constant.

Mất tương đương nhưng giữ equisatisfiability, đủ cho refutation.

### Unification

**Tìm substitution  $\theta$  sao cho  $E_1\theta = E_2\theta$ . MGU là substitution tổng quát nhất. Constant khác nhau hoặc function khác tên/arity  $\rightarrow$  fail. Occurs check: x không thể unify với  $f(x)$ .**

Ví dụ:  $\text{Knows}(\text{John}, x)$  và  $\text{Knows}(y, \text{Mother}(y))$  có thể cần giải ràng buộc đồng thời; luôn áp dụng substitution mới lên toàn bộ substitution đang có, không giải từng vị trí độc lập.

### FOL resolution

- Standardize variables apart giữa hai clause; tìm literal đối ngẫu có thể unify; ghi MGU  $\theta$ ; bỏ cặp literal và áp dụng  $\theta$  lên resolvent.
- Lặp đến  $\square$ . Trên bài thi, ghi dạng:  $C1, C2, \theta = \{...\} \Rightarrow C3$ .

Bẫy lớn: thiếu standardize-apart, bỏ quên  $\theta$ , dùng sai Skolem function, hoặc dịch sai 'only'.

## 5. Minimax và Alpha-Beta Pruning | ƯU TIÊN CAO

Đọc ở slide: Lecture 5 - Adversarial Search. Minimax trang 18-21; complexity trang 22; alpha-beta trang 26-32; cutoff/evaluation trang 33-38; stochastic games trang 39-45.

### Minimax

**MAX node lấy max giá trị child; MIN node lấy min. Time  $O(b^m)$ , space  $O(bm)$  khi duyệt sâu. Kết quả giả định đối thủ chơi tối ưu.**

## MAX-VALUE

```
v =  $-\infty$ 
for child:
v = max(v, MIN-VALUE(child))
return v
```

MAX lưu  $\alpha$ : cận dưới tốt nhất MAX đã đảm bảo được.

## MIN-VALUE

```
v =  $+\infty$ 
for child:
v = min(v, MAX-VALUE(child))
return v
```

MIN lưu  $\beta$ : cận trên tốt nhất MIN đã đảm bảo được.

## Alpha-Beta - điều kiện cắt

**Tại MAX:** sau khi cập nhật  $v$ , nếu  $v \geq \beta$  thì prune; ngược lại  $\alpha = \max(\alpha, v)$ .  
**Tại MIN:** sau khi cập nhật  $v$ , nếu  $v \leq \alpha$  thì prune; ngược lại  $\beta = \min(\beta, v)$ .

## Trình tự làm bài

- Duyệt DFS đúng thứ tự child để cho. Truyền  $(\alpha, \beta)$  từ cha xuống. Sau mỗi child: cập nhật  $v$ , kiểm tra prune ngay, rồi mới sang child kế.
- Ghi  $\alpha, \beta$  cạnh node; khoanh giá trị node; gạch toàn bộ nhánh chưa duyệt khi cắt. Root chọn action có minimax value tốt nhất.

## Bẫy

- Alpha-beta không đổi minimax value, chỉ giảm số node duyệt. Chỉ child chưa duyệt mới bị prune.
- Best move ordering có thể đạt khoảng  $O(b^{(m/2)})$ : hiệu quả phụ thuộc thứ tự duyệt.

Cutoff minimax: dừng ở depth limit/cutoff test và dùng EVAL thay UTILITY. Hàm EVAL tốt phải tương quan với kết quả thực và giữ đúng thứ tự win > draw > loss.

# 6. Constraint Satisfaction Problems | CSP + BACKTRACKING + AC-3

Đọc ở slide: Lecture 6. Mô hình trang 7-14; AC-3 trang 24-29; backtracking/MRV/LCV trang 30-38; FC vs AC trang 38-40; quiz trang 43-47; min-conflicts trang 48-53.

Mô hình CSP = Variables  $X$ , Domains  $D$ , Constraints  $C$ . Trong constraint graph: node là biến; edge nối hai biến có constraint chung.

## Backtracking

**Nếu assignment đầy đủ -> solution. Chọn biến chưa gán -> chọn giá trị -> nếu consistent thì gán và recurse -> thất bại thì undo/backtrack.**

| Heuristic | Chọn gì?                                           | Mục đích                    |
|-----------|----------------------------------------------------|-----------------------------|
| MRV       | Biến có domain còn ít giá trị nhất                 | Fail sớm                    |
| Degree    | Tie-break: biến ràng buộc nhiều biến chưa gán nhất | Tác động lớn nhất           |
| LCV       | Giá trị loại ít lựa chọn của hàng xóm nhất         | Giữ đường sống cho bước sau |

## Forward Checking

- Sau khi gán  $X=v$ , xóa khỏi domain hàng xóm các giá trị conflict.
- Nếu domain nào rỗng -> backtrack.
- Chỉ lan từ biến vừa được gán; nhẹ hơn AC-3.

## AC-3

- Queue ban đầu chứa các arc.
- REVISE( $X_i, X_j$ ): xóa  $x \in D_i$  nếu không tồn tại  $y \in D_j$  thỏa constraint.
- Nếu  $D_i$  rỗng -> failure. Nếu thay đổi, thêm lại  $(X_k, X_i)$ ,  $k \neq j$ .

## AC-3 pseudocode rút gọn

```
queue <- all arcs
while queue not empty: pop ( $X_i, X_j$ ); if REVISE( $X_i, X_j$ ): if  $D_i$  empty return failure; add ( $X_k, X_i$ )
for neighbors  $X_k \neq X_j$ 
return success
```

Khi làm bài: sau mỗi assignment/revise, ghi domain mới. Đừng chỉ ghi 'consistent'. Với Sudoku: biến = ô trống; domain = các số hợp lệ; constraint AllDiff theo hàng, cột, box.

## 7. Perceptron, MLP và Backpropagation | DỰ PHÒNG THEO SLIDE

Đọc ở slide: Lecture 11. Perceptron trang 14-22; MLP/backprop trang 24-29; gradient descent trang 34-37; momentum và biến thể trang 38-44; quiz trang 45-47.

### Perceptron

$$\text{net} = \sum_i w_i x_i - \theta; y = \text{step}(\text{net})$$

$$e = y_d - y; \Delta w_i = \eta x_i e; w_i \leftarrow w_i + \Delta w_i$$

- Bias có thể viết bằng  $x_0=1$  với weight  $w_0$ . Giữ đúng convention dấu bias/threshold mà đề cho.
- Perceptron một lớp không biểu diễn được XOR; MLP có hidden layer thì được.

### Sigmoid và forward pass

$$\sigma(z) = 1/(1+e^{-z}); \sigma'(z) = \sigma(z)(1-\sigma(z))$$

$$\text{net}_j = \sum_i w_{ij} x_i; y_j = \sigma(\text{net}_j)$$

$$\text{SSE: } E = 1/2 \sum_k (y_{d,k} - y_k)^2$$

### Backprop với sigmoid

$$\text{Output: } \delta_k = y_k(1-y_k)(y_{d,k} - y_k)$$

$$\text{Hidden: } \delta_j = y_j(1-y_j) \sum_k w_{jk} \delta_k$$

$$\Delta w_{ij} = \eta \delta_j x_i; w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$$

### Thứ tự tính

| Bước            | Việc cần làm                                    |
|-----------------|-------------------------------------------------|
| 1. Forward      | Tính net và output từng layer, giữ đủ số lẻ     |
| 2. Output delta | Tính error rồi nhân đạo hàm activation          |
| 3. Hidden delta | Dùng weight cũ và delta layer sau               |
| 4. Update       | Cập nhật mọi weight và bias sau khi đã có delta |

Bấy: cập nhật weight quá sớm rồi dùng weight mới để tính hidden delta; quên bias; đảo dấu (target-output); sai chiều index weight.

## 8. Uninformed và Informed Search | NỀN TẢNG

Đọc ở slide: Lecture 3 Part 1 - formulation trang 12-14, search structures trang 34-41; Part 2 - BFS 8-21, UCS 24-41, DFS 43-48, DLS 50-52, IDS 54-57, bidirectional 59-60; Part 3 - Greedy 12-18, A\* 20-36, heuristics/dominance 57-60.

| Thuật toán | Frontier / ưu tiên | Complete                    | Optimal          | Bẫy chính                 |
|------------|--------------------|-----------------------------|------------------|---------------------------|
| BFS        | FIFO               | Có, b hữu hạn               | Có nếu cost đều  | Tốn bộ nhớ                |
| DFS        | LIFO               | Không với depth vô hạn      | Không            | Dễ đi sai nhánh sâu       |
| UCS        | min g(n)           | Có nếu cost $\geq \epsilon$ | Có               | Goal test khi pop         |
| Greedy     | min h(n)           | Tùy graph/implementation    | Không            | Không nhìn cost đã đi     |
| A*         | min f=g+h          | Có trong điều kiện chuẩn    | Có với h phù hợp | Phải cập nhật path rẻ hơn |

### Heuristic

**Admissible:**  $0 \leq h(n) \leq h^*(n)$ , không overestimate chi phí thật đến goal.  
**Consistent:**  $h(n) \leq c(n, n') + h(n')$  cho mọi cạnh. Consistent  $\Rightarrow$  admissible (với  $h(\text{goal})=0$ ). A\* graph search optimal khi h consistent.

## Khi mô phỏng search

- Ghi rõ node gồm state, parent, action, g, depth. Tách thứ tự expanded khỏi returned path.
- Mỗi vòng: lấy node đúng cấu trúc frontier; kiểm tra goal theo convention của thuật toán; expand; xử lý duplicate; cập nhật frontier.
- UCS/A\*: dừng khi goal được lấy khỏi priority queue, không dừng ngay lúc goal vừa được sinh.
- Nếu tìm được đường rẻ hơn đến node đang ở frontier, cập nhật/reinsert node đó. Tie-break đúng như đề quy định.

Công thức nhớ nhanh: Greedy chỉ h; UCS chỉ g; A\* dùng g+h.

## 9. PEAS, Agent Types và Local Search | ĐỌC NHANH

Đọc ở slide: Lecture 2 - PEAS trang 26-42, agent types trang 49-64. Lecture 4 - hill climbing 10-19, simulated annealing 21-22, local beam 24-25, genetic algorithm 27-38.

### PEAS

**P = Performance measure; E = Environment; A = Actuators; S = Sensors.** Khi lập PEAS, mô tả đúng task environment cụ thể, không ghi tên thuật toán thay cho actuator/sensor.

| Agent              | Chọn hành động dựa trên                   | Điểm phân biệt                                   |
|--------------------|-------------------------------------------|--------------------------------------------------|
| Simple reflex      | Percept hiện tại + condition-action rules | Không lưu trạng thái                             |
| Model-based reflex | Internal state + model                    | Xử lý partially observable                       |
| Goal-based         | Trạng thái mục tiêu                       | Cần search/planning                              |
| Utility-based      | Expected utility                          | So sánh trade-off/rủi ro                         |
| Learning agent     | Experience/feedback                       | Performance, learning, critic, problem generator |

### Local search

| Thuật toán          | Ý tưởng                             | Bẫy / điểm nhớ                                  |
|---------------------|-------------------------------------|-------------------------------------------------|
| Hill climbing       | Chuyển sang neighbor tốt hơn        | Local maximum, ridge, plateau                   |
| Simulated annealing | Đôi lúc nhận bước xấu theo xác suất | Temperature giảm dần; thoát local optimum       |
| Local beam          | Giữ k trạng thái tốt nhất           | Các beam có thể hội tụ giống nhau               |
| Genetic algorithm   | Selection, crossover, mutation      | Representation và fitness quyết định chất lượng |

Các chủ đề này ít xuất hiện trong 3 bộ đề đã nhắc, nhưng là nguồn câu lý thuyết ngắn/trắc nghiệm. Ưu tiên sau khi làm chắc ID3, logic, minimax, NB và CSP.

## 10. Checklist trước khi thi | CHỐT ĐIỂM

| Chủ đề      | Tự kiểm trong 2 phút                                                                              |
|-------------|---------------------------------------------------------------------------------------------------|
| ID3         | Viết được H, AE, IG, GR; nhớ 3 điều kiện dừng; tự dựng 1 cây                                      |
| Naive Bayes | Viết score; Laplace đúng mẫu số; giải thích vì sao bỏ P(X)                                        |
| PL          | Đổi $\rightarrow$ , $\leftarrow$ ; đưa CNF; refutation bắt đầu bằng $\neg$ query                  |
| FOL         | Dịch lượng từ; standardize; Skolem; MGU; resolution có $\theta$                                   |
| Alpha-Beta  | MAX: $v \geq \beta$ ; MIN: $v \leq \alpha$ ; cập nhật sau từng child                              |
| CSP         | MRV, Degree, LCV; FC vs AC-3; REVISE đúng chiều                                                   |
| Neural net  | Forward $\rightarrow$ output delta $\rightarrow$ hidden delta $\rightarrow$ update; giữ weight cũ |
| Search      | BFS/DFS/UCS/Greedy/A*; goal test UCS/A* khi pop; h consistent                                     |

## Cách trình bày để lấy điểm

- Viết công thức trước rồi mới thay số; giữ phân số hoặc ít nhất 4 chữ số thập phân ở bước giữa.
- Logic: đánh số clause; ghi clause cha và substitution. Alpha-beta: ghi  $\alpha, \beta$  tại node. CSP: ghi domain sau từng revise.
- Neural net: tách net  $\rightarrow$  activation  $\rightarrow$  error  $\rightarrow$  update. Search: ghi frontier/expanded order nếu đề yêu cầu.
- Nếu kẹt, vẫn ghi quy trình đúng và các đại lượng trung gian; đây thường là phần điểm lớn.

## Lộ trình chắc cú

| Vòng | Mục tiêu                                                                 |
|------|--------------------------------------------------------------------------|
| 1    | ID3 + PL/FOL: làm lại toàn bộ câu đề cũ                                  |
| 2    | Minimax + Naive Bayes + CSP: giải không nhìn tài liệu                    |
| 3    | Neural net + search + PEAS/local search: ôn công thức và câu lý thuyết   |
| 4    | Làm một đề đủ thời gian; chấm lỗi trình bày; dùng cheatsheet rà lần cuối |

Ưu tiên độ chính xác hơn tốc độ: mỗi lần làm sai, ghi lỗi vào cheatsheet cá nhân bằng một dòng ngắn.